

操作系统

第二章 进程管理

第二部分 (2) 系统调用

同济大学计算机系



Part 1、系统调用基础

- 系统调用是为内核保留的一个中断。应用程序执行系统调用，促使执行它的进程陷入内核，执行语义确切的内核服务逻辑。Unix V6++和Linux，中断号是0x80。
- 系统调用是软件中断。是CPU执行int指令或syscall指令向自己发送的中断请求。 Unix V6++和Linux，用int 0x80指令。
- 每个系统调用独用一个系统调用号，用户空间有一个钩子函数，内核空间有一个入口函数。
 - 用户空间的钩子函数在标准C库。Unix V6++，src/lib/Lib_V6++.a；源代码在src/lib/src。比如 fork 系统调用在sys.c：
`int fork( )`
 - 内核空间的入口函数，Unix V6++在 src/interrupt/SystemCall.cpp，统一定义为：

`int SystemCall::Sys_*****()`

入口地址，登记在系统调用入口表，参见 src/include/SystemCall.h 和 src/interrupt/SystemCall.cpp。



一、用户空间 钩子函数

- 每个系统调用有一个钩子函数。
- 钩子函数为系统调用 (1) 准备系统调用号和入口参数 (2) 引发中断 (3) 接收系统调用的返回结果。
 - 执行成功，返回非负值。
 - 执行失败，返回 -1。
- 每个应用程序有一个全局变量 `errno`，是错误码，钩子函数用它存放系统调用的报错信息。正整数表示出错，0 表示执行成功。
- 运行在 i386 架构中的操作系统。陷入内核前，用 `EAX` 寄存器存放系统调用号，其余通用寄存器存放系统调用的入口参数 (`Unix V6++`, `EBX`、`ECX`、`EDX`、`ESI` 和 `EDI`)；执行完毕，`EAX` 存放系统调用的返回值。



例1: 20#系统调用 getpid()

```
#include <stdio.h>
#include <stdlib.h>           // src/lib/include/stdlib.h。定义有 errno
```

```
int main1(int argc, char* argv[])
{
1:  int i = getpid( );  // 应用程序执行getpid( )，获取执行该程序的进程的pid号
2:  if( i >= 0 )
3:      printf( " My pid is %d\n ", i );
4:  else
5:      printf( " Fail System Call ! return value = %d, errno = %d \n ", i , errno );
}
```

```
int getpid()
{
1:  int res;
2:  _asm__volatile ( "int $0x80": "=a"(res): "a"(20) );
3:  if ( res >= 0 )
4:      return res;
5:  else
6:  {
7:      errno = -res ;
8:      return -1;
9:  }
} // 系统调用的钩子函数
```

指令部

输出部

输入部

内联汇编语句的执行过程：

1：输入部：提供系统调用号，**eax = 20**。

2、指令部：**int 0x80**。CPU执行指令，响应**0x80**号中断请求：CPU陷入内核执行系统调用，取现运行进程 **p_pid**，存入**EAX**寄存器。系统调用返回，返回地址是输出部首地址。

3、输出部：读取**EAX**寄存器，获得进程**p_pid**，赋值**res**变量。



例2：带参数的 3#系统调用 read

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");
    int fd1 = open(argv[1],O_RDONLY);
    int fd2 = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(fd1,&c,1) == 1 )
        write(fd2,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

./copy oldfile newfile

```
int read(int fd, char* buf, int nbytes)
{
    int res;
    __asm__ __volatile__ ("int $0x80":"=a"(res):"a"(3),"b"(fd),"c"(buf),"d"(nbytes));
    if ( res >= 0 )
        return res;
    return -1;
}
```

2、陷入内核

1、输入部： **eax** = read系统调用号3
ebx = 第1个参数，oldFile的文件描述符
ecx = 第2个参数，内存变量c，用来存放从oldFile读入的数据
edx = 第3个参数，希望读入的字节数，1

3、输出部：
read系统调用读入的字节数 → **res**



二、内核对系统调用的处理 SystemCall类

```
class SystemCall                                // SystemCall.h
{
public:
    static const unsigned int SYSTEM_CALL_NUM = 64;

public:
    static void SystemCallEntrance();    // 系统调用总入口
    static void Trap(struct pt_regs* regs, struct pt_context* context);    // 系统调用处理函数
    static void Trap1(int (*func)());    // 驱动系统调用入口函数

private:    // 系统调用入口函数
    static int Sys_NullSystemCall();    // 没有0#系统调用
    static int Sys_Rexit();    // 1#
    static int Sys_Fork();    // 2#
    static int Sys_Read();
    static int Sys_Write();
    static int Sys_Open();
    static int Sys_Close();    ..... 共计48个已定义的系统调用

private:
    static SystemCallTableEntry m_SystemEntranceTable[SYSTEM_CALL_NUM];    // 系统调用入口函数表
};
```

```
struct SystemCallTableEntry
{
    unsigned int count;    // 参数数量
    int (*call)();    // 系统调用入口函数
};
```



```
SystemCallTableEntry SystemCall::m_SystemEntranceTable[SYSTEM_CALL_NUM] =
```

```
{ // 系统调用入口表 System.cpp
    { 0, &Sys_NullSystemCall },
    { 1, &Sys_Rexit },
    { 0, &Sys_Fork },
    { 3, &Sys_Read },
    { 3, &Sys_Write },
    { 2, &Sys_Open },
    { 1, &Sys_Close },
    .....
    { 0, &Sys_Nosys},
    ..... // 共计 64个元素
};
```

```
enum ErrorCode
```

```
{ // 系统调用的出错码 User.h
    NOERROR = 0, /* No error */
    .....
    ENOENT = 2, /* open出错，文件不存在 */
    .....
    EINTR = 4, /* 系统调用被信号打断，提前结束 */
    .....
    ENOMEM = 12, /* Out of memory */
    EACCES = 13, /* open出错，文件无访问权限 */
    ..... // 共计32个元素
};
```


1、在内核和用户空间传递参数 和 返回值

```
void SystemCall::SystemCallEntrance()
```

```
{
    SaveContext();
    SwitchToKernel();
    CallHandler(SystemCall, Trap);
```

```
L:    例行调度
    if(RunRun)
        Swtch( );
```

```
    RestoreContext();
    Leave();
    InterruptReturn();
}
```

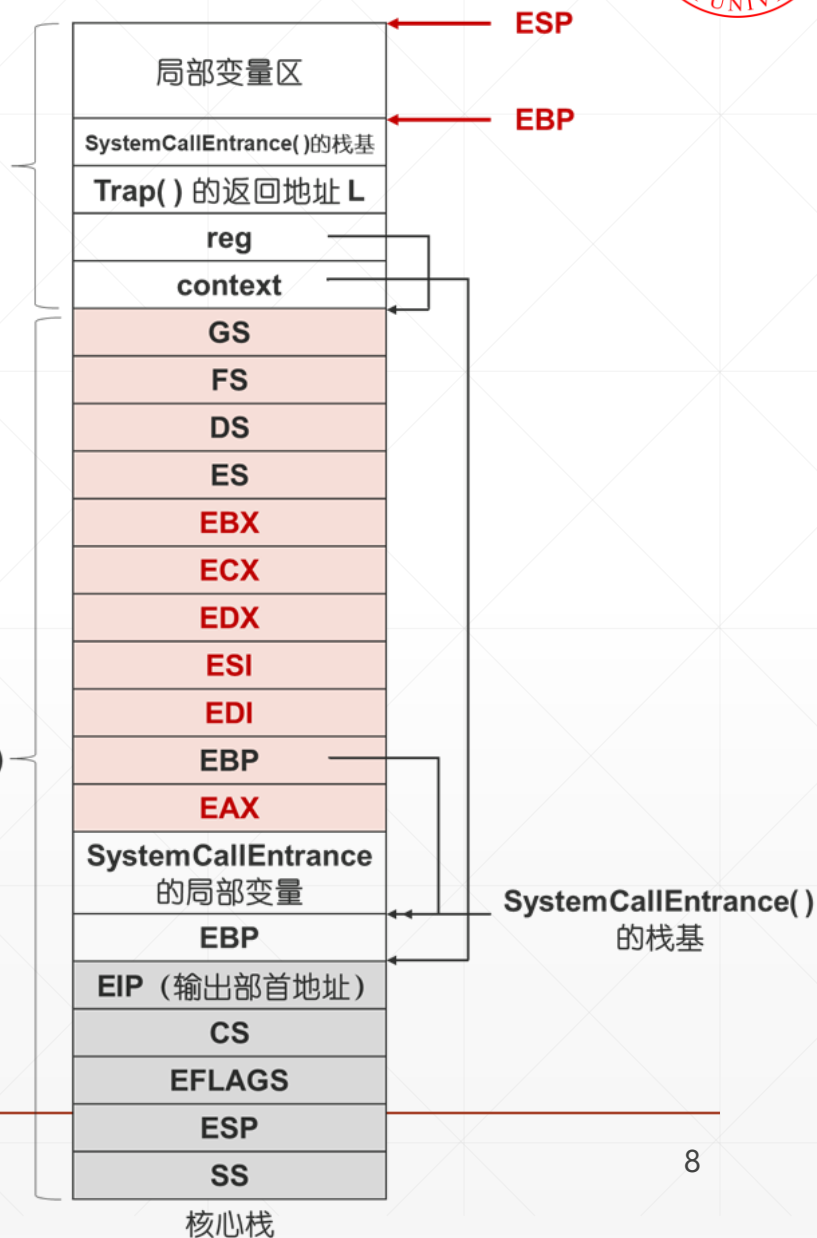
1、执行系统调用前，压栈保护用户态现场：
reg->eax，系统调用号
reg->ebx ~~~，参数

2、系统调用执行时，将返回值存入：
reg->eax

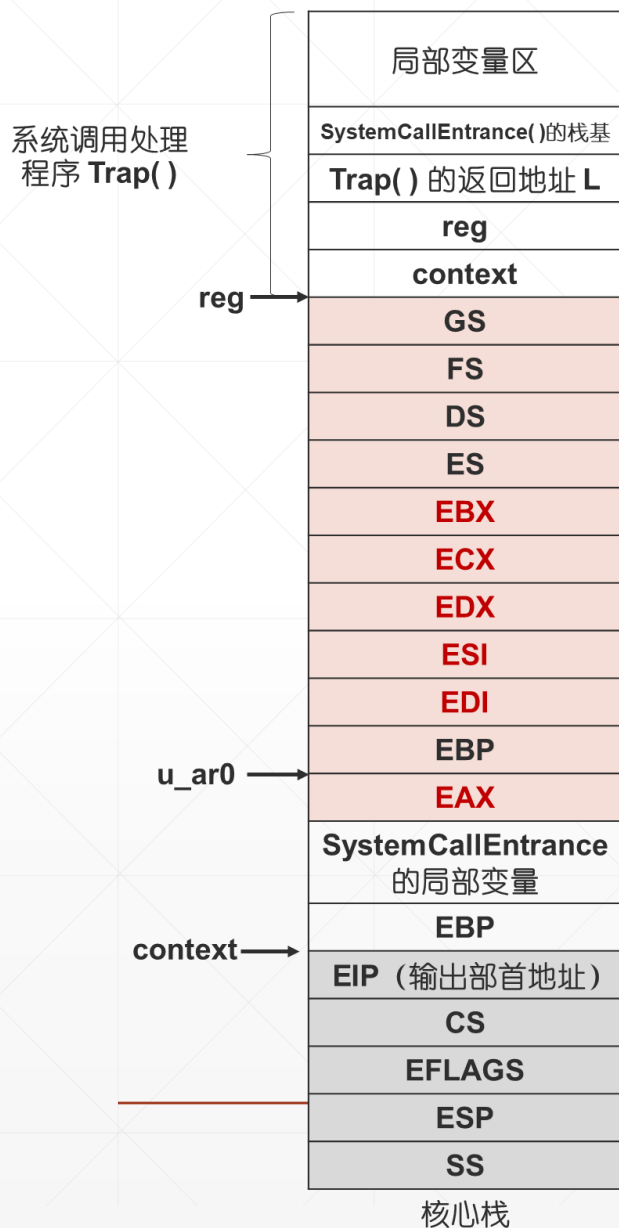
3、系统调用返回时，弹出所有寄存器，恢复用户态现场。
寄存器 EAX 就是返回值。

系统调用处理程序 Trap()

系统调用入口程序 SystemCallEntrance()



2、系统调用使用的全局变量



`u_arg` 数组

第1个参数
第2个参数
.....
.....
第5个参数

`class User`

{

public:

`static const int EAX = 0;`

.....

`unsigned int *u_ar0; // 存系统调用返回值的单元`

`int u_arg[5]; // 参数`

`bool u_intflg; // True表示系统调用被信号打断 (执行期间收到信号)`

`ErrorCode u_error; // 错误码`

.....

}

3、Trap(), 驱动系统调用

Trap()

{

1. 用系统调用号查系统调用入口表, 得入口函数首地址和参数数量
2. 取系统调用参数, 存入全局变量 u_arg
3. u_ar0定位 "EAX"
- 4. Trap1() 驱动系统调用**
5. 系统调用有被信号打断 ?
有, u_error = User::EINTR
6. 系统调用成功 ?
否, u_ar0[EAX] = -u_error
7. 信号处理, 令进程返回用户态后先执行信号处理程序
8. SetPri(), 计算进程执行应用程序时的优先数
9. 返回总入口函数 SystemCallEntrance()

}

Trap1()

{

1. u_error = User::NOERROR
2. u_intflg = 1
3. SaveU(u.u_qsav)
- 4. 调用系统调用入口函数**
5. u_intflg = 0

}

4、系统调用执行

Trap1()

{

1. u_error = User::NOERROR

2. u_intflg = 1

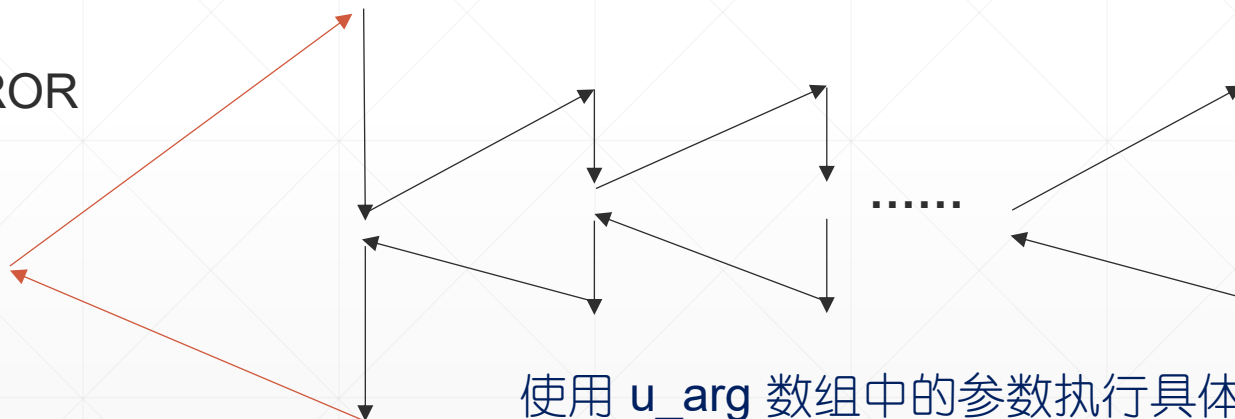
3. SaveU(u.u_qsav)

4. 调用系统调用入口函数

5. u_intflg = 0

}

Sys_*()**



使用 u_arg 数组中的参数执行具体的系统调用操作

- 成功, u_ar0[EAX] = 返回值

- 失败, u_error = 出错码

如果被信号打断, RetU(u.u_qsav)

例2: getpid() 的内核实现

1、20#系统调用

2、系统调用入口表，登记入口地址和参数数量

```
SystemCallTableEntry SystemCall::m_SystemEntranceTable[SYSTEM_CALL_NUM] =  
{ // 系统调用入口表 System.cpp  
    .....  
    { 0, &Sys_Getpid}, // 第21个元素  
    .....  
};
```

3、写系统调用函数

```
/*      20 = getpid      count = 0      */  
int SystemCall::Sys_Getpid()  
{  
    User& u = Kernel::Instance().GetUser();  
    u.u_ar0[User::EAX] = u.u_procp->p_pid;  
    return 0; /* GCC likes it ! */  
}
```

实验三、系统调用

- 为 Unix V6++ 添加49#系统调用 `get_ppid`, 获得父进程的pid号
- 为 Unix V6++ 添加50#系统调用 `get_pids`, 获得自己 和 父进程的pid号
- 为 Unix V6++ 添加51#系统调用 `get_proc`, 获得进程Process结构和User结构中的多个字段, 比如:
 - 虚空间: 代码段起始地址, 代码段长度, 数据段起始地址, 数据段长度, 堆栈段长度
 - 物理空间: 代码段起始地址, 可交换部分起始地址
- 以最后一小题为背景, 评论 CISC、RISC架构 的优势 和 劣势

三、模式切换

- 用户态→核心态
 - 现运行进程从执行 应用程序的状态 转变为 执行内核代码的状态
 - 响应中断
- 核心态→用户态
 - IRET

四、为什么模式切换

- 执行以下操作需要操作系统特权
 - 执行IO操作
 - 读取、设置时钟
 - 访问除通用寄存器（含EFLAGS）之外的其它寄存器
 - 执行特权指令。比如，开中断 STI、关中断 CLI
 - 执行内核函数，访问内核变量

所以，执行前，现运行进程要模式切换，提升自己的资源访问权限。



Part 2、系统调用 和 入睡、唤醒

- **不会入睡的系统调用。**比如，getpid。进程陷入内核，读取内核变量p_pid
- **快系统调用**会导致进程进入高优先级睡眠状态SSLEEP。是一些会访问磁盘的系统调用，比如 open、read、write 磁盘文件
 - 特征 (1) 进程睡眠时长，内核可控。不存在入睡后，很久无法唤醒的情形
 - (2) 入睡期间，进程不响应信号。
- **慢系统调用**会导致进程进入低优先级睡眠状态SWAIT，比如 sleep(50000000)，读键盘 (scanf) ，读socket, P操作.....
 - 特征 (1) 进程睡眠时长，完全不由内核控制。有可能入睡后，永远无法唤醒。
 - (2) 为了提高并行系统效率，进程入睡期间响应信号。待信号处理完毕后，再行入睡等待操作。



进程的优先级 1

- UNIX进程的优先级动态变化，除非正在执行系统调用，否则

进程优先数 $p\text{-pri} = \min \{ 255 \text{ (127)} , \text{静态优先数} + (p\text{-cpu}/16) \}$

- 静态优先数 = $PUSER + p\text{-nice}$ ；是一个相对稳定的值。 $PUSER == 100$ ， $p\text{-nice}$ 的默认值是0
 - 应用程序可以用nice系统调用调节进程的 $p\text{-nice}$ 值。后台进程 `nice(20)`，将自己的 $p\text{-nice}$ 调成20，把算力让给前台作业。
- $p\text{-cpu}/16$ 是动态因子，根据进程使用CPU的情况调节优先数。密集使用CPU， $p\text{-cpu}$ 上升，优先级下降；很久未运行， $p\text{-cpu}$ 变小，优先级上升。这个因子用来实现应用程序轮流使用CPU。时钟中断处，详述。

进程的优先级 2

- UNIX V6, 所有系统调用优先级固定。
 - 不会入睡的进程优先数不变, 维持原值
 - 入睡进程, 设置p_pri = 这个系统调用的优先数
 - 优先数 $\in [0, 100)$, SWAIT, 低优先权睡眠
 - 优先数 $\in [-100, 0)$, SSLEEP, 高优先权睡眠

高优先级的系统调用先执行, 有利于提高资源周转速度, 提升系统性能。

表 5.2: UNIX V6++中定义的睡眠进程拥有的优先数

序号	名称	优先数值	睡眠原因
1	PSWP	-100	0#进程等待执行换入/换出任务
2	PINOD	-90	分配/释放 <u>Inode</u> (文件控制块)
3	PRIBIO	-50	read/write 系统调用读写磁盘文件
4	EXPRI	-1	同时执行 exec 的进程数量过多, 等待其它进程 exec 系统调用执行完毕
5	PPIPE	1	读写管道文件
6	TTIPRI	10	read/write 系统调用等待键盘输入
7	TTOPRI	20	暂时未用到
8	PWAIT	40	父进程执行 wait 系统调用, 等待子进程终止
9	PSLEP	90	设置定时器入睡
10	PUSER	100	无

入睡

```
void Process::Sleep(unsigned long chan, int pri)
```

```
{
```

```
    User& u = Kernel::Instance().GetUser();
```

```
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
```

```
    if ( pri > 0 )
```

```
    { .....

```

```
        X86Assembly::CLI();
```

```
        this->p_wchan = chan;
```

```
        this->p_stat = Process::SWAIT;
```

```
        this->p_pri = pri;
```

```
        X86Assembly::STI();
```

```
        .....

```

```
        Kernel::Instance().GetProcessManager().Swch();
```

```
        .....

```

```
    } // ....., 中级调度和信号处理逻辑
```

```
    else {
```

```
        X86Assembly::CLI();
```

```
        this->p_wchan = chan;
```

```
        this->p_stat = Process::SSLEEP;
```

```
        this->p_pri = pri;
```

```
        X86Assembly::STI();
```

```
        Kernel::Instance().GetProcessManager().Swch();
```

```
    }
```

```
}
```

唤醒 1

```
void ProcessManager::WakeUpAll(unsigned long chan)
{
    for(int i = 0; i < ProcessManager::NPROC; i++)
    {
        if( this->process[i].IsSleepOn(chan) )
        {
            this->process[i].SetRun();
        }
    }
} // 遍历Process数组，唤醒睡眠原因等于chan的进程
```

```
bool Process::IsSleepOn(unsigned long chan)
{
    if( this->p_wchan == chan && ( this->p_stat == Process::SWAIT || this->p_stat == Process::SSLEEP ) )
        return true;
    return false;
} // 检查睡眠原因，是否为chan
```



唤醒 2

```
void Process::SetRun()
{
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();

    /* 清除睡眠原因，转为就绪状态 */
    this->p_wchan = 0;
    this->p_stat = Process::SRUN;
    if ( this->p_pri < procMgr.CurPri )
    {
        procMgr.RunRun++; // 置抢占标识
    }
    if ( 0 != procMgr.RunOut && (this->p_flag & Process::SLOAD) == 0 )
    {
        procMgr.RunOut = 0;
        procMgr.WakeUpAll((unsigned long)&procMgr.RunOut);
    } // 中级调度逻辑
}
```



会入睡的系统调用 read

应用程序copy

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
L: while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

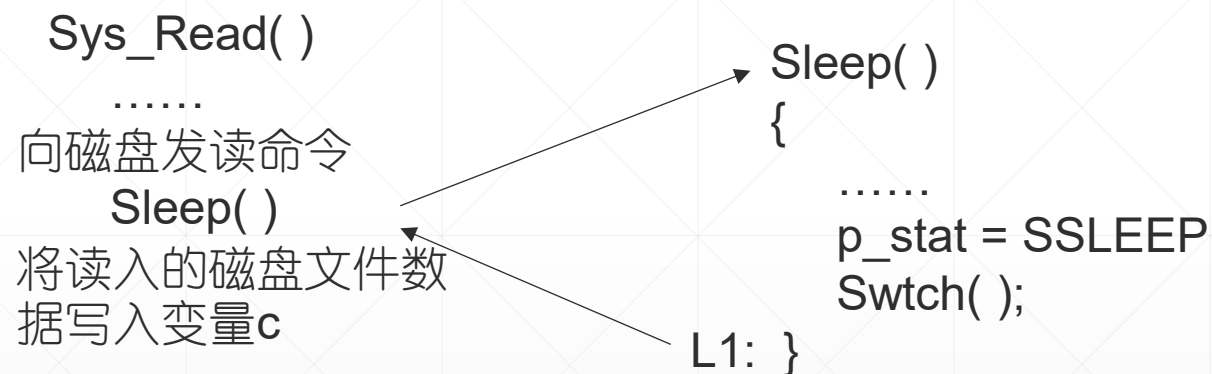
    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

内核kernel

..... → Sys_Read() 读取磁盘文件中的数据，送变量c

Sys_Read()的具体执行过程



- (1) `copy`进程向外设发IO命令
- (2) 执行`Sleep()`函数入睡，放弃CPU
- (3) IO完成后，`copy`进程被唤醒
- (4) 得到CPU的使用权，`Sleep()`返回
- (5) 将读入的磁盘数据送用户空间。

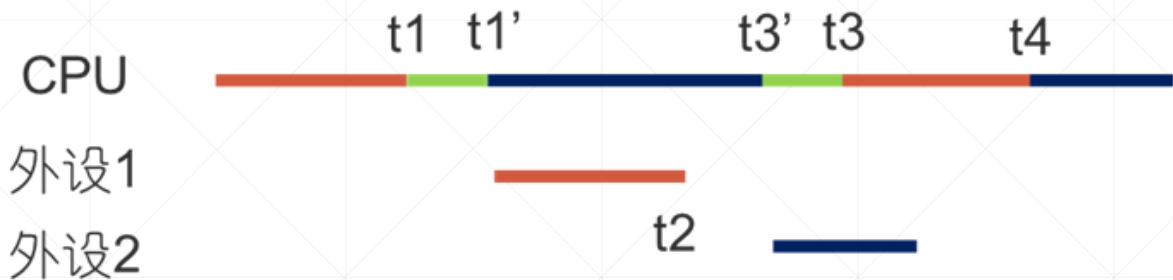
(1,2) 是系统调用上半段
(3,4,5)是系统调用下半段

Sys_Read(), 情景分析



- copy进程, t1时刻, 执行read系统调用上半段。t1' 时刻, 磁盘IO, 放弃CPU。
- t2时刻, IO操作完成。现运行进程PA, 执行磁盘中断处理程序, 唤醒copy进程。
- t3' 时刻, copy进程被调度到 (因为PA IO放弃CPU), 执行read系统调用下半段。
- t3时刻, read系统调用运行结束, copy进程返回用户态, 执行应用程序, 处理内核送来的文件数据。

t1 → t1' 系统调用上半段

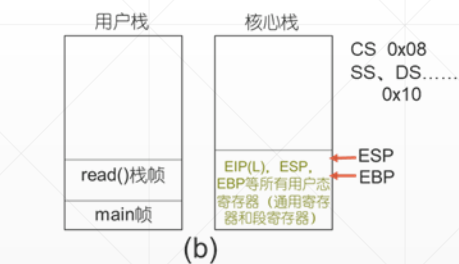
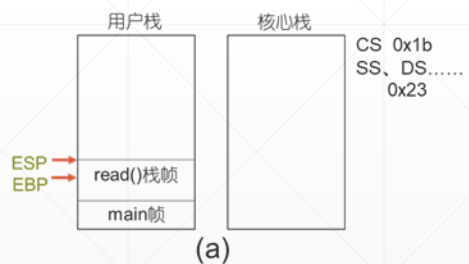


read系统调用
.....
向磁盘发IO命令
Sleep()
将读入数据送变量c

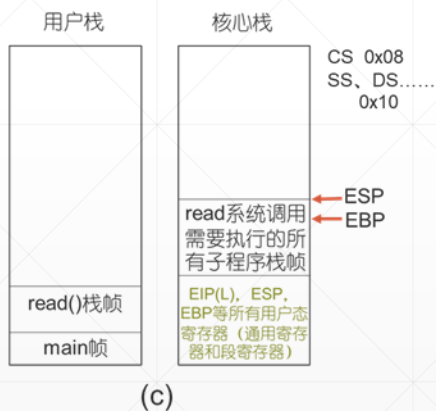
Sleep()
{
.....
p_stat = SSLEEP
Switch();
L1: }

t1', 进程切换

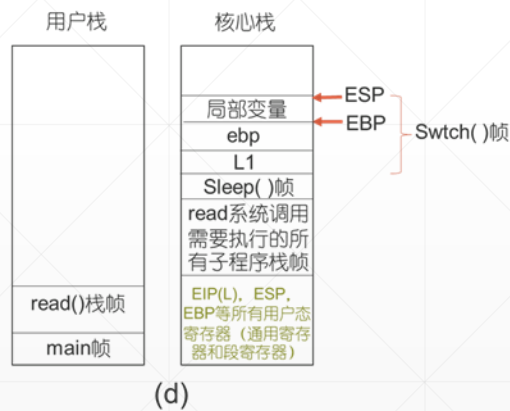
t1, 模式切换



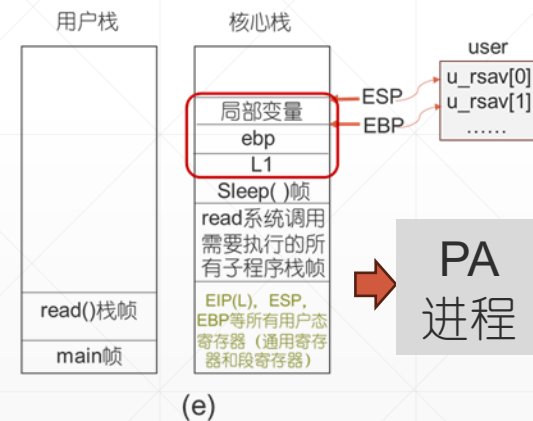
存用户态寄存器
换核心态段选择子



执行系统调用



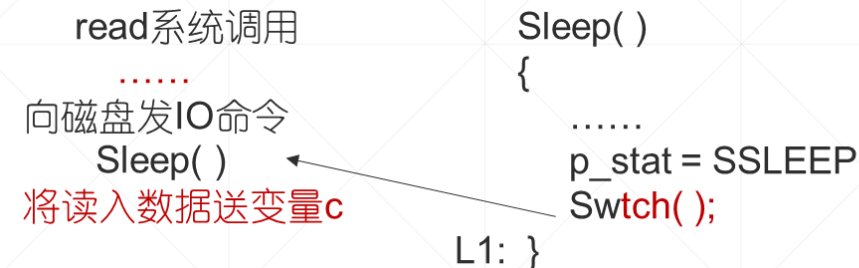
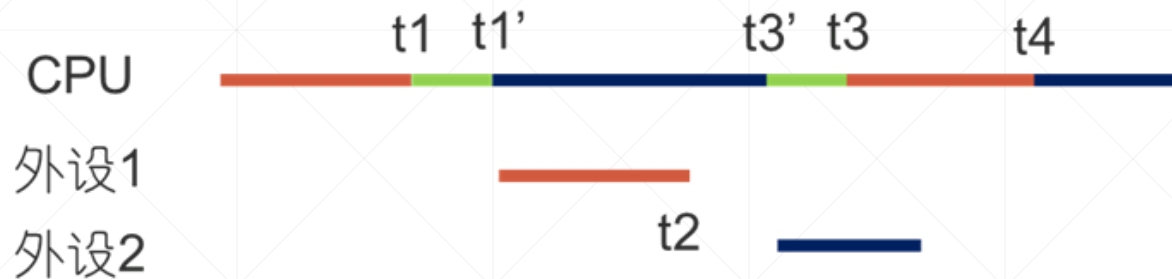
发IO命令后
入睡放弃CPU



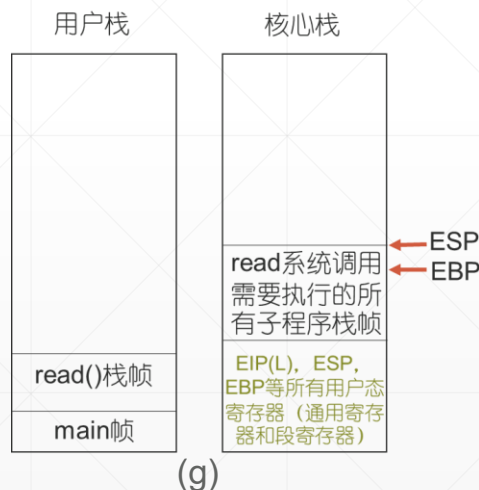
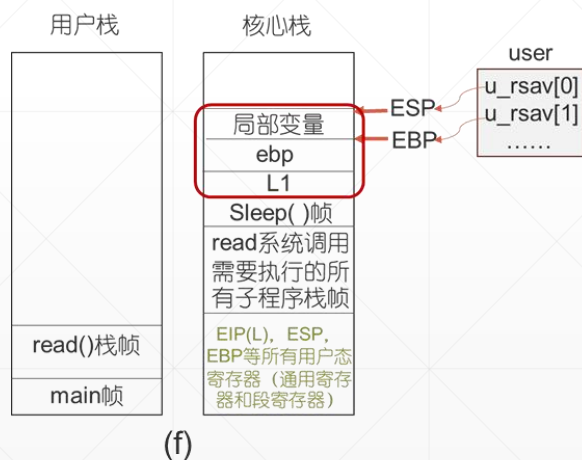
存Switch()返回地址
& 栈帧的ESP、EBP

PA
进程

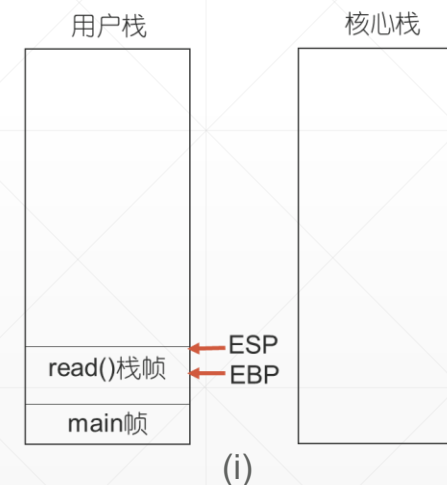
t3' → t3 系统调用下半段



t3', 进程切换



t3, 模式切换



恢复Swth() 栈帧的ESP、EBP
函数返回, 建立地址映射关系

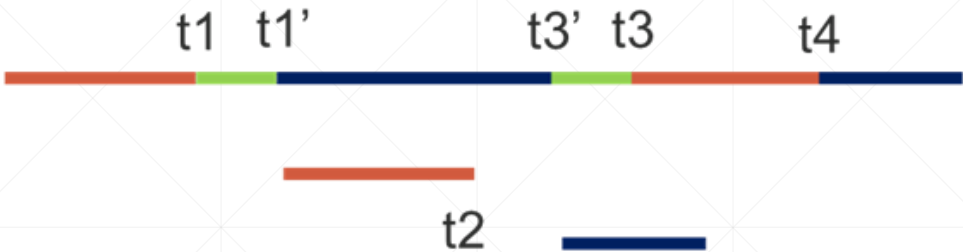
继续执行系统调用, 送磁盘
文件数据至用户空间, 变量c

系统调用执行完毕
恢复用户态寄存器

继续执行应用程序



copy进程从放弃CPU至恢复执行经历2个Swtch()函数执行实例



t1', copy进程启动的Swtch实例

```
int ProcessManager::Swtch() // 现运行进程执行Swtch( )函数放弃CPU
{
    {
        User& u = Kernel::Instance().GetUser(); // 0、u 值为0xC03ff000的指针，指向现运行进程的user结构
        SaveU(u.u_rsav); // 1、将 esp 和 ebp 存入user结构的u_rsav字段

        Process* procZero = &process[0]; // 2、0#进程成为现运行进程
        X86Assembly::CLI();
        SwtchUStruct(procZero);
        RetU();
        X86Assembly::STI();
        Process* selected = Select(); // 2、遍历Process数组，挑选内存中优先级最高的可执行进程 selected

        X86Assembly::CLI();
        SwtchUStruct(selected); // 3、为selected进程的PPDA区建立地址映射关系
        RetU(); // 4、访问其 user结构，从u_rsav中取出保存的esp和ebp，恢复selected进程的Swtch栈帧
        X86Assembly::STI();
        User& newu = Kernel::Instance().GetUser(); // 5、利用newu指针获得selected进程的相对表，
        newu.u_MemoryDescriptor.MapToPageTable(); // 为CPU建立该进程用户空间的地址映射关系

        .....

        return 1; // 6、新运行进程selected从Swtch返回
    }
}
```

t3', PA进程启动的Swtch实例

```
int ProcessManager::Swtch() // 现运行进程执行Swtch( )函数放弃CPU
{
    {
        User& u = Kernel::Instance().GetUser(); // 0、u 值为0xC03ff000的指针，指向现运行进程的user结构
        SaveU(u.u_rsav); // 1、将 esp 和 ebp 存入user结构的u_rsav字段

        Process* procZero = &process[0]; // 2、0#进程成为现运行进程
        X86Assembly::CLI();
        SwtchUStruct(procZero);
        RetU();
        X86Assembly::STI();
        Process* selected = Select(); // 2、遍历Process数组，挑选内存中优先级最高的可执行进程 selected

        {
            X86Assembly::CLI();
            SwtchUStruct(selected); // 3、为selected进程的PPDA区建立地址映射关系
            RetU(); // 4、访问其 user结构，从u_rsav中取出保存的esp和ebp，恢复selected进程的Swtch栈帧
            X86Assembly::STI();
            User& newu = Kernel::Instance().GetUser(); // 5、利用newu指针获得selected进程的相对表，
            newu.u_MemoryDescriptor.MapToPageTable(); // 为CPU建立该进程用户空间的地址映射关系

            .....

            return 1; // 6、新运行进程selected从Swtch返回
        }
    }
}
```



Part 3、CPU空转



Process* ProcessManager::Select ()

```
{
    static int lastSelect = 0; // Process数组的搜索起点，是上次被选中的进程的Process数组下标。初值。0

    while (true)
    {
        int priority = 256; int best = -1; this->RunRun = 0;
        for ( int count = 0; count < NPROC ; count++ ) // 遍历Process数组，从lastSelect的下一个数组元素开始
        {
            int i = (lastSelect + 1 + count) % NPROC;
            if ( Process::SRUN == process[i].p_stat && (process[i].p_flag & Process::SLOAD) != 0 )
            {
                // SRUN状态，图像在内存中，优先数最小（优先级最高）的进程
                if ( process[i].p_pri < priority )
                {
                    best = i;    priority = process[i].p_pri;
                }
            }
        }

        if ( -1 == best )
        {
            __asm__ __volatile__ ("hlt");
            continue;
        }

        this->CurPri = priority;
        lastSelect = best;
        return &process[best];
    }
}
```

Process数组





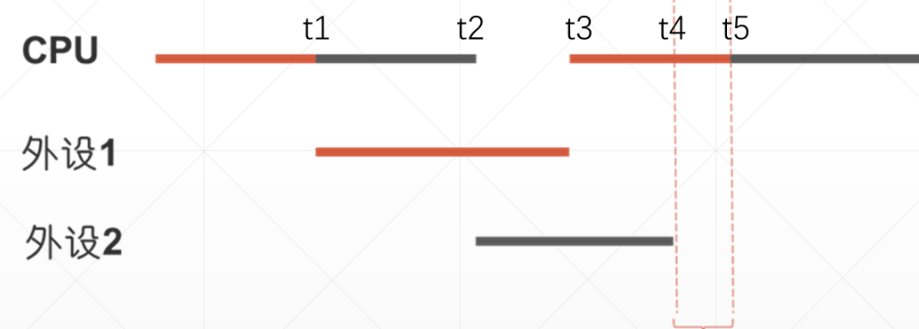
Process* ProcessManager::Select ()

```
{
    static int lastSelect = 0; // Process数组的搜索起点，是上次被选中的进程的Process数组下标。初值。0

    while (true)
    {
        int priority = 256; int best = -1; this->RunRun = 0;
        for ( int count = 0; count < NPROC ; count++ ) // 遍历Process数组，从lastSelect的下个数组元素开始
        {
            int i = (lastSelect + 1 + count) % NPROC;
            if ( Process::SRUN == process[i].p_stat && (process[i].p_flag & Process::SLOAD) != 0 )
            {
                // SRUN状态，图像在内存中，优先数最小（优先级最高）的进程
                if ( process[i].p_pri < priority )
                {
                    best = i;    priority = process[i].p_pri;
                }
            }
        }

        if ( -1 == best )
        {
            __asm__ __volatile__ ("hlt");
            continue;
        }

        this->CurPri = priority;
        lastSelect = best;
        return &process[best];
    }
}
```



t2时刻，现运行进程PB入睡放弃CPU后，系统没有就绪进程（SRUN），best == -1。
0#进程执行hlt指令，CPU进入低功耗状态，不动。
CPU idle，等待出现就绪进程。

t3时刻，0#进程响应中断，唤醒进程PA。
随后，PA被选中运行。